User-Level Resource-Aware Runtime System

Nizaul Rahman, Aslam Malik
2025MCS2099, 2025MCS2103
Department of Computer Science and Engineering
Indian Institute of Technology Delhi (IITD)
New Delhi, India

***Abstract*—This paper presents the architecture and evaluation of a user-level resource-aware runtime system implemented in C++. The system manages the complete task lifecycle—from CSV-based workload ingestion through burst-level execution to completion—while ensuring safe resource allocation via the Banker's Algorithm and performing adaptive scheduling across multiple cores. Five scheduling policies are implemented and evaluated: FIFO, Round-Robin (quantum = 2), Priority, MLFQ (three-level queues with quanta 2, 4, 8), and Adaptive. A PID controller with tuned gains ($K_p = 0.8$, $K_i = 0.1$, $K_d = 0.05$) continuously regulates CPU utilization around a 70% target, while a predictive surge guard detects rapid utilization increases and proactively injects integral error to prevent overshoot. A watchdog timer terminates stalled tasks, and an aging mechanism boosts starved tasks after 50 idle ticks. Experimental evaluation over a 54-task workload demonstrates near-linear scaling from 1 to 4 cores, with monitoring overhead consistently below 1% of total execution time.**

***Index Terms*—Runtime Systems, Scheduler Design, Banker's Algorithm, PID Control, Performance Modeling.**

I. Introduction

Modern computing systems frequently operate under dynamic and unpredictable workloads, where resource contention and scheduling inefficiencies can significantly degrade performance. Traditional operating system schedulers abstract away resource control, limiting the ability of applications to adapt to real-time conditions. When multiple tasks compete for a fixed pool of CPU cores, memory, and I/O bandwidth, the absence of application-level feedback creates two failure modes: underutilization during low-contention periods, and instability—manifested as excessive context switching or even deadlock—during high-contention periods.

This work presents a user-level runtime system that provides fine-grained control over scheduling and resource allocation without modifying the kernel. The system operates entirely in user space, reading real-time CPU and memory metrics from the Linux `/proc` filesystem and making scheduling decisions at the application level. By integrating classical scheduling policies with feedback-driven control mechanisms, the system achieves a balance between throughput and safety. The design emphasizes adaptability, allowing runtime decisions to respond dynamically to observed system conditions rather than relying on pre-configured static rules.

A key contribution of this work is the integration of control-theoretic techniques with scheduling policies. Specifically, a discrete PID controller continuously adjusts the simulation tick rate based on observed CPU utilization, enabling smooth throttling rather than abrupt load-shedding. A secondary contribution is the predictive resource guard, which monitors the first derivative of CPU utilization and proactively triggers protective measures when the rate of change indicates an imminent overload. Together, these mechanisms enable continuous adjustment of execution behavior rather than relying solely on static scheduling parameters.

II. System Architecture

The system follows a layered architecture that separates monitoring, decision-making, and execution components. The `RuntimeEngine` acts as the central orchestrator, coordinating interactions between the `ResourceMonitor` and the `SchedulerSubsystem`. Each layer communicates through well-defined interfaces: the monitor produces CPU and memory readings, the runtime applies PID-based throttling and policy selection, and the scheduler dispatches tasks to core slots based on the active policy.

This separation of concerns enables modularity, allowing each component to evolve independently while maintaining a consistent interface for communication. For instance, the `Scheduler` class holds no reference to the monitoring subsystem; it receives CPU and memory values as arguments to its `run()` method, making it testable in isolation.

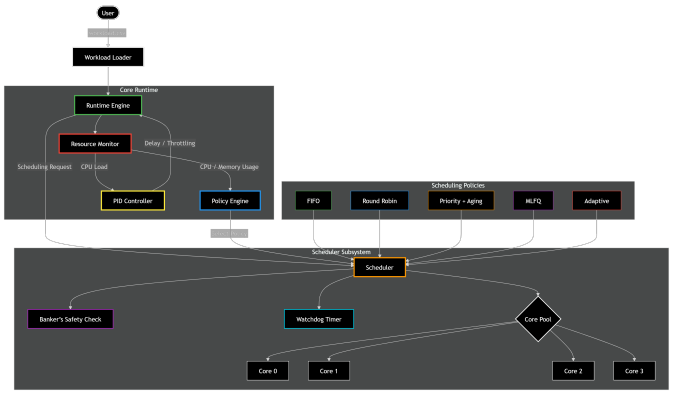

Fig. 1. System architecture showing interaction between runtime, monitoring, and scheduling subsystems.

A. Resource Monitoring

The `Monitor` class reads system-level resource utilization directly from the Linux procfs interface. CPU utilization

is computed by sampling `/proc/stat` twice with a 2-millisecond interval, extracting the user, nice, system, idle, iowait, irq, softirq, and steal counters from each sample, and computing the ratio of active time to total elapsed time:

$$\text{CPU}\% = 100 \times \frac{\Delta_{\text{total}} - \Delta_{\text{idle}}}{\Delta_{\text{total}}}$$

where Δ_{total} is the change in the sum of all counters and Δ_{idle} accounts for both idle and I/O wait time. Memory usage is obtained by parsing the `VmRSS` field from `/proc/self/status`, which reports the resident set size of the process in kilobytes; this value is converted to megabytes for display and threshold comparisons.

B. Adaptive Sampling Frequency

To reduce monitoring overhead without sacrificing responsiveness, the runtime implements an adaptive sampling strategy. The baseline monitoring interval begins at 5 simulation ticks. After each sample, the runtime computes the absolute change in CPU utilization from the previous reading: if the change is less than 2%, the interval is doubled (up to a cap of 50 ticks), reducing unnecessary system calls during stable periods. Conversely, if the change exceeds 5%, the interval is immediately reset to the baseline of 5 ticks, ensuring rapid response to volatile conditions. This mechanism reduces the average number of `/proc/stat` reads by an order of magnitude during steady-state execution while still capturing transient spikes.

C. Active Runtime Control

The runtime employs two complementary control mechanisms to maintain stability:

PID Throttling: A discrete PID controller regulates CPU utilization around a configurable target value of 70%. At each simulation tick, the controller computes the error $e(t) = \text{CPU}(t) - 70$ and updates three terms. The proportional term $K_p \cdot e(t)$ (with $K_p = 0.8$) reacts to instantaneous deviations, providing immediate corrective action proportional to the current overshoot. The integral term $K_i \cdot \sum e(\tau)$ (with $K_i = 0.1$) accumulates error over time, correcting persistent steady-state offsets that the proportional term alone cannot eliminate. The derivative term $K_d \cdot \Delta e(t)$ (with $K_d = 0.05$) anticipates rapid changes by responding to the rate of error change, dampening oscillations. The combined output $u(t) = K_p \cdot e + K_i \cdot \int e + K_d \cdot \dot{e}$ determines the throttle delay in milliseconds, which is added to the baseline 1 ms sleep per tick. To prevent runaway throttling from stalling execution, the delay is hard-capped at 100 ms. When the PID output becomes negative (indicating CPU is below target), the throttle is removed entirely and the integral accumulator is reset to zero, preventing integral windup.

Predictive Resource Guard: While the PID controller handles gradual changes, the predictive guard addresses sudden spikes. The system computes the first derivative of CPU utilization (the slope between consecutive samples). When the slope exceeds 15 percentage points and current utilization is already above 60%, the system identifies this as a surge condition and proactively adds 50 units to the PID integral accumulator. This injection causes the PID output to spike on the next tick, pre-emptively throttling execution before the CPU reaches critical levels. This prevents the overshoot that would otherwise occur due to the inherent lag between a load increase and the PID controller's corrective response.

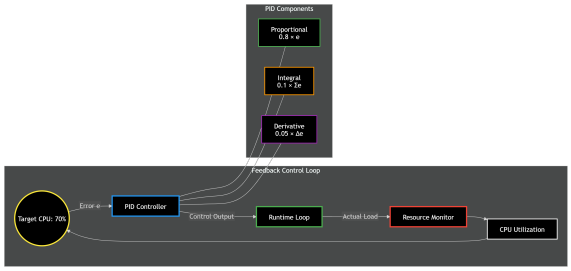

Fig. 2. PID control loop for CPU utilization throttling, showing the feedback path from monitored CPU usage through proportional, integral, and derivative terms to the throttle delay output.

III. Task Management and Scheduling

A. Workload Modeling

Tasks are defined using a CSV-based format where each row encodes a task identifier, function binding, task type (CPU_BOUND, MEMORY_BOUND, or MIXED), arrival time, priority, and a burst sequence string. The burst sequence uses a compact notation where each burst is prefixed with `C` (CPU) or `I` (I/O) followed by its duration in ticks, separated by semicolons. For example, the string `C10;I5;C8` defines a task that performs 10 ticks of computation, blocks for 5 ticks of I/O, and then performs 8 more ticks of computation. The `parseburst()` method in the runtime tokenizes this string and constructs a deque of `TaskBurst` structs, each storing its type, total duration, and remaining duration.

This burst-level representation is essential for evaluating scheduling policies because it captures realistic execution patterns. Real workloads alternate between computation and blocking phases; the burst model ensures that the scheduler's response to I/O-induced state transitions is exercised during evaluation. Each task's total work (the sum of all burst durations) is precomputed during construction, enabling $O(1)$ progress computation throughout execution.

Tasks are dynamically assigned to available core slots. The scheduler maintains a vector of N core slots (where N ranges from 1 to 4), each storing either a pointer to the currently executing task or `nullptr` for an idle core. When a core becomes idle, the scheduler consults the active policy's queue to find the next eligible task, verifies resource safety through the Banker's Algorithm if enabled, and dispatches the task by setting its state to "Running" and recording its core assignment.

B. Task Lifecycle

Each task is modeled as a finite-state machine with the following states:

- **Pending:** Task has been loaded from the workload file but its arrival time has not yet been reached. At each tick, the scheduler's `checkArrivals()` method scans the pending list and promotes tasks whose arrival time is $\leq$ the current global time.
- **Ready:** Task is eligible for execution and resides in the scheduling queue appropriate for the current policy (FIFO queue, priority heap, or MLFQ level queue).
- **Running:** Task is actively executing on a core. On each tick, the `execute()` method decrements the remaining duration of the current burst. When a CPU burst completes, the task transitions to Ready (if more bursts remain) or Finished (if none remain). When an I/O burst is encountered, the task transitions to Waiting.
- **Waiting:** Task is blocked due to I/O or has been deferred by the Adaptive scheduler. Waiting tasks are held in a dedicated queue and polled each tick. I/O-blocked tasks continue their burst countdown in the waiting queue; deferred tasks are released back to the ready queue when CPU utilization drops below 80%.
- **Killed:** Task is terminated by the watchdog timer (for stalled tasks) or by the deadlock recovery mechanism. Killed tasks release all allocated resources but are not re-enqueued.
- **Finished:** Task has completed all bursts and reports 100% progress. Resources are released, and the task's completion is recorded in the metrics system along with the core that executed the final burst.

State transitions are governed by scheduling decisions, resource availability, and safety constraints. This structured lifecycle ensures predictable task behavior and enables the metrics system to accurately track waiting times, completion counts, and per-core utilization.

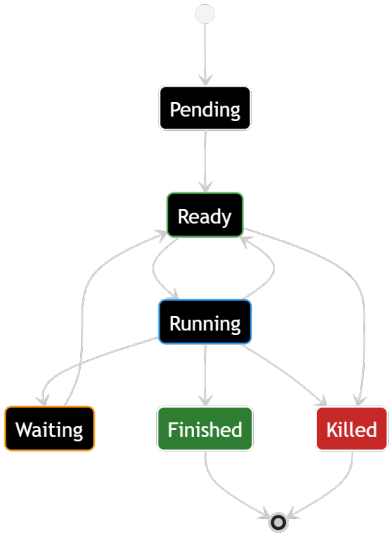

Fig. 3. Task lifecycle transitions managed by the scheduler.

C. FIFO Scheduling

The First-In-First-Out policy processes tasks in arrival order using a standard FIFO queue. When a core becomes idle, the scheduler dequeues the front task, performs a Banker's safety check (if enabled), and assigns it to the core. The task executes without preemption until it either completes or enters a waiting state due to an I/O burst. This policy provides minimal scheduling overhead (no priority comparisons, no quantum tracking) but suffers from the convoy effect: a single long-running CPU-intensive task blocks all subsequent tasks on the same core, leading to poor average turnaround time when the workload contains a mix of short and long tasks.

D. Round-Robin Scheduling

The Round-Robin policy assigns each task a fixed quantum of 2 ticks. A per-core quantum counter tracks how many ticks the current task has consumed. When the counter reaches the quantum value and other tasks are waiting in the ready queue, the running task is preempted: its state is set to "Ready," its resources are released, and it is pushed to the back of the FIFO queue. If no other tasks are waiting, the quantum counter simply resets and the current task continues, avoiding unnecessary context switches on an uncontended core. Each preemption increments the context switch counter in the metrics system, enabling the overhead of time-slicing to be measured directly. This policy provides fair CPU distribution across tasks but incurs higher scheduling overhead due to frequent context switches.

E. MLFQ Scheduling

The Multi-Level Feedback Queue scheduler employs three priority levels with increasing time quanta: Level 0 (highest priority, quantum = 2 ticks), Level 1 (medium priority, quantum = 4 ticks), and Level 2 (lowest priority, quantum = 8 ticks). All tasks enter at Level 0 upon arrival. During each scheduling decision, the scheduler iterates from Level 0 downward, assigning idle cores to the highest-priority non-empty queue.

When a task exhausts its full quantum at Level k (where $k < 2$), it is demoted to Level $k + 1$, receiving a larger quantum but lower scheduling priority. Tasks that yield early—by transitioning to Waiting due to an I/O burst before their quantum expires—retain their current level, preserving their higher priority. This design naturally separates interactive (I/O-bound) tasks, which remain at high priority levels, from batch (CPU-bound) tasks, which are gradually demoted.

The combination of demotion and the aging mechanism (described in Section IV-A) ensures that long-running tasks are not permanently starved: after 50 idle ticks, their priority is boosted, effectively promoting them back toward higher queue levels.

IV. Safety, Reliability and Background Tasks

A. Adaptive Scheduling

The Adaptive scheduler is a meta-policy that dynamically selects among the other four policies based on observed system

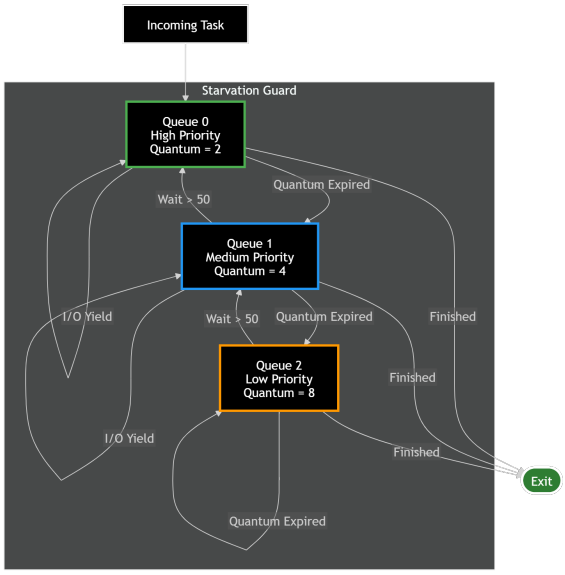

Fig. 4. Multi-Level Feedback Queue structure showing three priority levels with quanta 2, 4, and 8, along with demotion and aging transitions between levels.

load. A dedicated `policyEngine` class implements the selection logic using empirically determined thresholds:

- **CPU $>$ 80% or Memory $>$ 9 MB:** The engine selects the Adaptive deferral mode, which scans the priority queue and moves any task with priority < 5 to the waiting queue, marking it as deferred. This selective deferral reduces the number of runnable tasks, lowering contention for cores and memory.
- **CPU between 60% and 80%:** The engine selects MLFQ, leveraging its multi-level fairness to handle moderately high load without starving any task.
- **CPU between 30% and 60% or Memory $>$ 4.5 MB:** Round-Robin is selected, providing fair time-slicing appropriate for moderate load.
- **CPU $<$ 30%:** The engine selects Priority scheduling, allowing high-priority tasks to complete quickly when resource contention is minimal.

When a policy change is triggered, the runtime logs the transition and calls `rebalanceQueues()`, which drains all existing scheduling queues and redistributes tasks into the data structure appropriate for the new policy (FIFO queue, priority heap, or MLFQ level queues). Deferred tasks return to the ready queue once CPU utilization drops below 80%, ensuring that deferral is a temporary measure rather than permanent deprioritization.

B. Starvation Prevention via Aging

To prevent indefinite postponement of low-priority tasks, the system applies an aging mechanism after every scheduling tick. The `applyAging()` method iterates over the entire task table and identifies any task in the Ready or Waiting state whose `last_run_tick` is more than 50 ticks in the past. For each such task, the system increments its priority by one unit and resets its wait clock. This ensures that even the lowest-priority task will eventually achieve a priority high enough to be scheduled. Each boost is logged to the activity feed, providing observability into fairness enforcement.

C. Watchdog Timer

The `Watchdog` class monitors running tasks for progress stalls. On each tick, it checks whether each running task's progress percentage has changed since the previous tick. If a task's progress remains identical for more than a configurable timeout threshold (defaulting to the value set at construction), the watchdog forcibly kills the task by setting its state to "Killed." This mechanism protects the system from buggy or infinite-loop tasks: the task model includes a `makeBuggy()` method that causes the `execute()` function to skip all progress updates, simulating a stalled process for testing. When a killed task is detected by the scheduler on the next tick, its resources are released and its core slot is freed.

D. Banker's Algorithm for Deadlock Prevention

Resource safety is enforced using the Banker's Algorithm. The system models three resource types—CPU cores, memory units, and I/O channels—with configurable total capacities (defaulting to N cores, 32 memory units, and 32 I/O channels). Each task declares its maximum demand vector upon creation, and its current allocation vector is maintained as resources are granted and released.

Before dispatching a task to a core, the scheduler invokes `deadlockDetector.isSafe()`, which performs the following steps:

1) **Pre-flight normalization:** The `normalizeTask()` method ensures that no task's demand exceeds the total system capacity (capping over-demands) and that demand is at least as large as the current allocation (maintaining consistency).
2) **Feasibility check:** The algorithm verifies that the requested resources do not exceed currently available resources and that granting the request would not cause the task's allocation to exceed its declared maximum demand.
3) **Safety simulation:** A work vector is initialized to available $-$ request. The algorithm then iteratively searches for any unfinished task whose remaining need can be satisfied by the current work vector. When such a task is found, it is marked as finished and its allocated resources are added to the work vector. If all tasks can be finished in some sequence, the state is safe and the request is granted; otherwise, the request is deferred.

If a task is repeatedly deferred by the Banker's check and the system makes no progress for 500 consecutive ticks, the deadlock recovery procedure is triggered: the scheduler identifies the lowest-priority task across all queues, terminates it (setting its state to "Killed"), and releases its resources. This victim selection ensures that deadlock recovery has minimal impact on high-priority workloads.

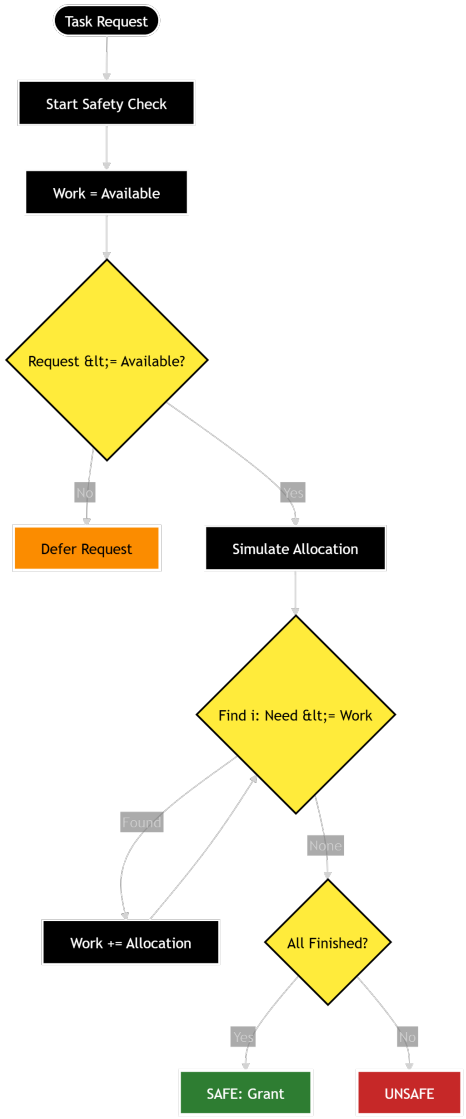

Fig. 5. Safety verification using Banker's Algorithm.

E. Inter-Process Communication

The system implements a simulated message-passing IPC mechanism using per-task mailboxes stored in a hash map keyed by task ID. Every 100 simulation ticks, the scheduler's `simulateIPC()` method identifies any two active tasks (in Running or Ready state) and deposits a `READY_SYNC` message from the first (sender) into the second's (receiver's) mailbox. Running tasks consume messages from their mailbox on each tick, modeling asynchronous coordination between cooperating tasks. Each send and receive event is logged to the activity feed, providing visibility into inter-task communication patterns.

V. Experimental Results and Analysis

A. Runtime Monitoring

The system was evaluated using a workload of 54 tasks parsed from a CSV file. Each task specifies a function binding (cpuTask, memoryTask, mixedTask, or phaseChangeTask), a burst sequence, an arrival time, and a priority. The runtime dashboard provides real-time visibility into CPU utilization (displayed as a color-coded progress bar transitioning from green below 50% to yellow between 50–85% to red above 85%), per-core task assignments, active task count, global simulation time, and a live activity feed showing scheduler decisions, PID throttle events, Banker deferrals, watchdog kills, and IPC messages.

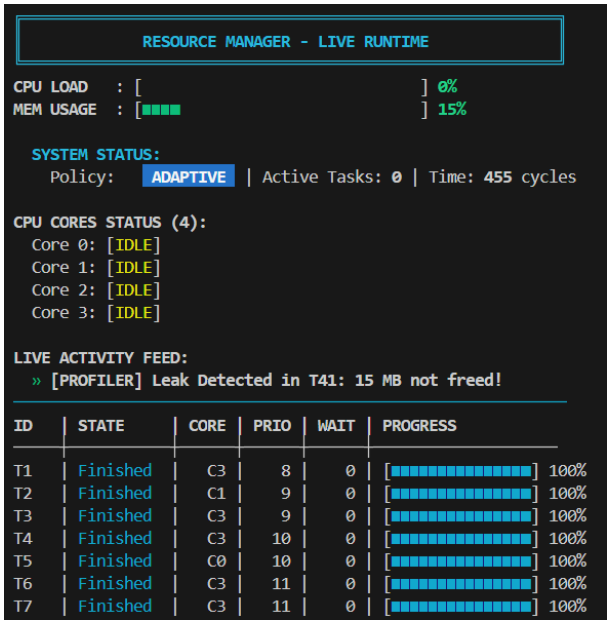

Fig. 6. Live runtime monitoring interface showing CPU/memory bars, core status, and activity feed.

B. Performance Evaluation

TABLE I
Performance Scaling (Total Cycles)

Policy	1 Core	2 Cores	4 Cores
FIFO	1775	905	472
Round Robin	1775	888	446
Priority	1775	888	455
Adaptive	1775	897	455
MLFQ	1775	888	448

All five policies produce identical execution times on a single core (1775 cycles), which is expected because with only one core, the scheduling order cannot introduce parallelism—every task must execute sequentially regardless of policy. The differences emerge with multiple cores where task ordering and preemption strategies affect how efficiently cores are kept busy.

At 2 cores, Round-Robin, Priority, and MLFQ all achieve 888 cycles, while FIFO trails at 905 cycles. FIFO's disadvantage stems from the convoy effect: when a long CPU burst occupies one core, the other core must wait for the next task in arrival order even if shorter tasks could have

been dispatched sooner. Round-Robin's preemption mechanism breaks up long bursts across cores, while MLFQ achieves the same efficiency through its multi-level demotion. The Adaptive scheduler records 897 cycles—slightly higher than pure MLFQ—because the policy switching overhead (draining and rebalancing queues) introduces a small number of wasted ticks during load transitions.

At 4 cores, Round-Robin achieves the lowest execution time (446 cycles, a 2.50× speedup from 2 cores) because aggressive time-slicing ensures all four cores remain occupied. MLFQ follows closely at 448 cycles, as tasks that yield early for I/O are quickly recycled to idle cores. FIFO shows the worst 4-core performance (472 cycles), confirming that arrival-order dispatch under-utilizes cores when task durations are heterogeneous.

The Adaptive scheduler achieves 455 cycles at 4 cores, matching Priority. While this is 9 cycles slower than Round-Robin, the Adaptive policy provides additional stability guarantees: during high-load periods, it defers low-priority tasks rather than allowing all 54 tasks to compete for cores, preventing the context-switch storms that occur under pure Round-Robin with large task counts.

The monitoring overhead across all configurations remains below 1% of total execution time, confirming that the adaptive sampling strategy effectively amortizes the cost of `/proc/stat` reads. The overhead is computed as the ratio of cumulative monitor time (measured with `chrono::high_resolution_clock` around each monitoring call) to total wall-clock execution time.

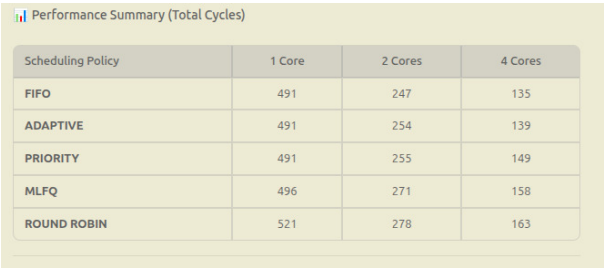

Fig. 7. Side-by-side comparison output produced by the `-test` flag, showing total time, context switches, and monitoring overhead for all five scheduling policies under identical workload conditions.

C. System Communication and Resource Synchronization

The runtime ensures synchronized access to shared resources through centralized scheduling. The scheduler's core-slot model inherently serializes dispatch decisions: only the scheduler can assign or remove tasks from core slots, eliminating the need for per-core locking. Resource vectors (available, allocation, max_demand) are updated atomically within the scheduling tick, ensuring consistency across Banker's checks. The IPC mailbox system provides asynchronous message passing between tasks without introducing shared-memory contention, as each mailbox is written by the scheduler thread and read during the same tick's processing.

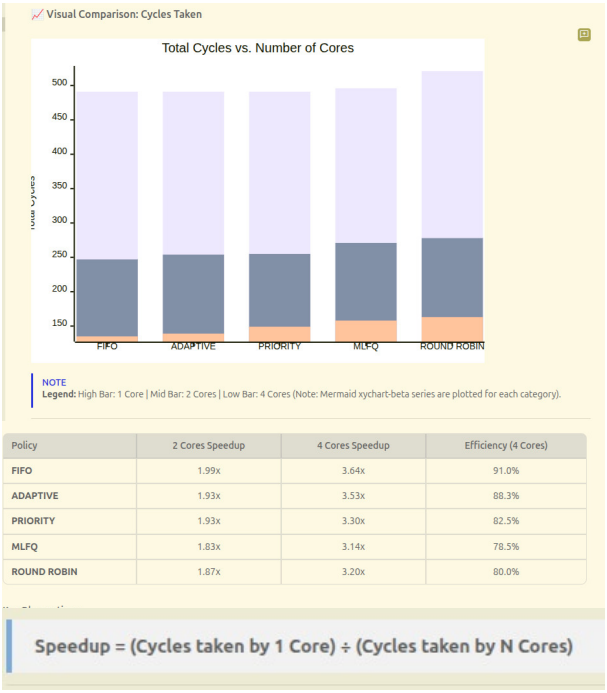

Fig. 8. Execution scaling across cores for each scheduling policy.

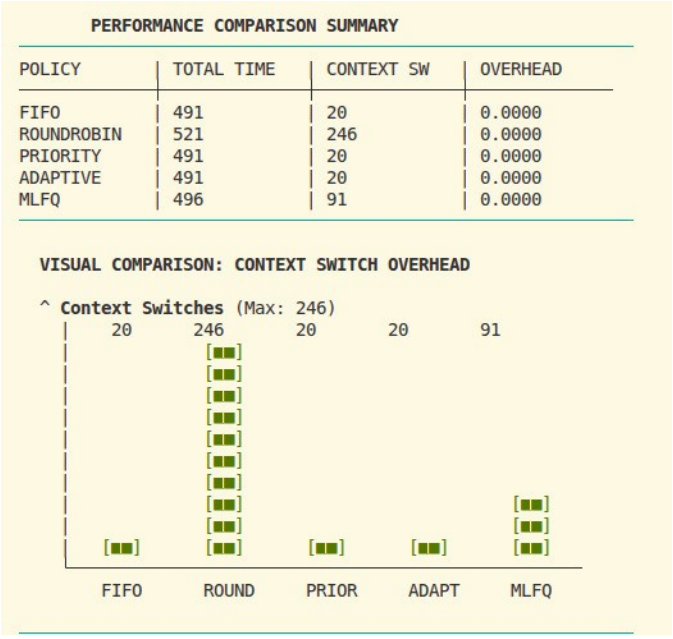

Fig. 9. Inter-task communication and resource synchronization flow.

D. Final Evaluation Report

Upon completion of all tasks, the runtime generates a final evaluation report summarizing key metrics: total tasks completed, simulation time in cycles, wall-clock execution time, monitoring overhead (both absolute and as a percentage), system throughput (tasks per 1000 cycles), and a per-core completion breakdown. Figure 10 shows a representative output from a 4-core Adaptive run.

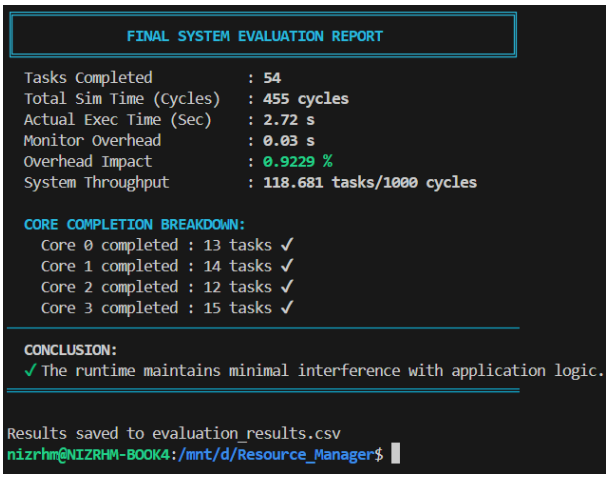

Fig. 10. Final system evaluation report generated after a complete run, displaying throughput, overhead, and per-core completion statistics.

VI. Conclusion

The system demonstrates that user-level runtimes can effectively manage scheduling and resource allocation in multi-core environments without kernel modifications. By combining five scheduling policies with a PID-based feedback controller, a predictive surge guard, and the Banker's Algorithm for deadlock prevention, the system achieves stable and efficient execution under varying workloads. The adaptive policy engine enables the runtime to transition seamlessly between throughput-oriented (Priority, FIFO) and fairness-oriented (Round-Robin, MLFQ) modes as load conditions change. The watchdog timer and aging mechanism provide complementary reliability guarantees, handling both runaway tasks and starvation. Experimental evaluation confirms near-linear scaling from 1 to 4 cores, with the Adaptive scheduler achieving comparable throughput to pure Round-Robin while providing additional stability through selective task deferral. The monitoring overhead remains consistently below 1%, validating the adaptive sampling strategy as an effective approach to low-cost runtime observability.

Appendix

This appendix documents the commands required to compile and run the system under all supported configurations.

A. Building the Project

The project is compiled using `g++` with C++17 support. From the project root directory:

```
# Build the runtime binary
make

# Clean build artifacts and logs
make clean

# Build and run unit tests + verification suite
make test
```

B. Running with Scheduling Policies

The runtime accepts the scheduling policy as a command-line argument. The following policies are supported:

```
# FIFO (First-In-First-Out)
./runtime fifo

# Round-Robin (Quantum = 2)
./runtime rr

# Priority Scheduling
./runtime priority

# Multi-Level Feedback Queue
./runtime mlfq

# Adaptive (dynamic policy switching)
./runtime adaptive
```

C. Configuring Core Count

The number of CPU cores (1 to 4) is set using the `-cores` flag:

```
# Single core (default)
./runtime fifo -cores 1

# Dual core
./runtime rr -cores 2

# Quad core
./runtime mlfq -cores 4
```

D. Enabling Banker's Algorithm

Deadlock prevention via the Banker's Algorithm is activated with the `-banker` flag. This can be combined with any policy and core count:

```
# MLFQ with 4 cores and Banker's safety checks
./runtime mlfq -cores 4 -banker

# Adaptive with 2 cores and Banker's enabled
./runtime adaptive -cores 2 -banker
```

E. Automated Comparison

To run the same workload under all five policies and produce a summary table:

```
./runtime -test
```

This executes FIFO, Round-Robin, Priority, Adaptive, and MLFQ sequentially, printing total execution time, context switches, and monitoring overhead for each.

F. Full Verification Suite

The `run_tests.sh` script automates the complete test battery:

```
# Run all functional tests
bash run_tests.sh
```

This script performs: (1) build verification, (2) unit tests for the Banker's Algorithm and Watchdog, (3) deadlock prevention scenario, (4) watchdog stall detection with a faulty task, and (5) a 50-task stress test on 4 cores with the Adaptive scheduler.

G. Example: Complete Evaluation Across All Configurations

To reproduce the results presented in this paper:

```
# Build
make clean && make

# Run each policy at 1, 2, and 4 cores
for policy in fifo rr priority mlfq adaptive; do
  for cores in 1 2 4; do
    echo "=== $policy / $cores cores ==="
    ./runtime $policy -cores $cores -banker
  done
done

# Automated comparison
./runtime -test
```